PricePing Product Resolution Architecture

Purpose: Define a cost-efficient, marketplace-aware architecture for resolving products from a user's product search or shared URL, while minimizing external API calls and providing a controlled fallback path when the cached/database result is not the product the user wants.

Core strategy: PricePing DB first → user confirmation/selection → marketplace-specific external resolver only when required.

1. Architecture Goals

• Minimize paid/external API calls by reusing products already resolved and stored in PricePing DB.

• Use the strongest available source for each marketplace instead of forcing one generic search provider to handle every marketplace.

• Return an exact or high-confidence product match, including title, canonical URL, image, price and marketplace identifiers.

• Allow the user to reject an existing result and trigger a fresh external search.

• Store successful external resolutions back into PricePing DB so future users benefit from the previous lookup.

• Keep the resolver marketplace-neutral so Amazon can later switch from DataForSEO to Amazon Creators API without changing the UI or core product model.

2. High-Level Architecture

```
User → Product Search / Shared URL → Product Resolution Service → PricePing DB → If sufficient/high-confidence result exists: return DB result → If user rejects result or DB has no suitable result: identify marketplace → Flipkart → Flipkart Direct API → Amazon → DataForSEO Amazon API → Myntra → DataForSEO Google Shopping → Normalize → Match/Validate → Return candidates/product → Persist successful resolution → PricePing DB
```

3. Marketplace Strategy

Marketplace	Primary source	Why	Fallback / Future
Flipkart	Direct Flipkart API	Already available; structured marketplace data and direct product information	None required initially
Amazon	DataForSEO Amazon API	Amazon-specific structured product data; avoids unreliable scraping	Amazon Creators API while Account Qualifi
Myntra	DataForSEO Google Shopping	No dedicated DataForSEO Myntra API; Google Shopping can surface Myntra offerings/products	Separate Myntra/direct affiliate/

4. Complete Request Flow

Step 1 — Input

The user enters a product name or shares a product URL. The system first extracts obvious marketplace/domain information, product identifiers such as ASIN where available, and a normalized search phrase.

Step 2 — PricePing DB lookup

Search the internal product catalog before calling any external provider. Candidate matching should consider marketplace, normalized title, brand, model, variant attributes, identifiers and optionally semantic similarity.

Step 3 — Decide whether the DB result is sufficient

If there is a high-confidence match, return it immediately. This is the cheapest and fastest path. If no sufficiently good result exists, show available candidates when appropriate and allow the user to indicate that the result is not what they want.

Step 4 — User rejection / fresh resolution

When the user says the product is not what they want, bypass the previous DB candidate for that request and call the marketplace-specific external resolver. This prevents repeatedly returning the same incorrect cached product.

Step 5 — Marketplace routing

Use deterministic routing whenever the marketplace is known from the URL/domain. If the input is only a product name, use the selected marketplace filter or search scope to decide which resolver(s) to invoke.

Step 6 — Normalize external response

Convert Flipkart, Amazon/DataForSEO and Google Shopping/DataForSEO responses into one internal ProductCandidate schema. The UI and database should not depend on provider-specific response formats.

Step 7 — Product matching and validation

Rank candidates using title similarity plus exact/structured checks for brand, model number, storage, RAM, size, generation and other variant attributes. Do not rely only on a search-engine relevance score. A title threshold such as ≥0.90 can be used as one gate, followed by stricter model/variant validation.

Step 8 — User selection when multiple candidates exist

If several candidates are plausible, return a small candidate list rather than silently selecting an uncertain product. The user can select the correct one.

Step 9 — Persist

Store the selected/validated product and its marketplace-specific identifiers in PricePing DB. This turns every successful external resolution into reusable internal data.

5. Provider-Specific Flows

Amazon

Preferred path today: DataForSEO Amazon API. For an Amazon URL, extract ASIN where possible and prefer an ASIN-based lookup. For product-name searches, use Amazon product search and rank returned candidates. Capture title, URL, image URL, price, ASIN and other available attributes. Once Amazon Creators API access becomes available, replace the external resolver implementation while retaining the same internal interface.

Flipkart

Use the existing direct Flipkart API. Because this is a first-party/direct marketplace integration already available to PricePing, it should take precedence over generic search providers. Normalize its response into the common product schema and persist it.

Myntra

Use DataForSEO Google Shopping because there is no dedicated DataForSEO Myntra product API in the selected architecture. Search Google Shopping, filter/rank candidates whose merchant/source is Myntra, validate the product title and variant, then persist the Myntra URL, image, price and identifiers available from the result.

6. Recommended Internal Interfaces

The resolver should expose a provider-independent contract:

```
resolve_product(input, marketplace, force_external=False) -> ProductResolution ProductResolution:
status marketplace selected_product candidates[] confidence source cached
```

Provider adapters:

```
ProductProvider ■■■ PricePingDBProvider ■■■ FlipkartAPIProvider ■■■ DataForSEOAmazonProvider ■■■
DataForSEOGoogleShoppingProvider
```

This abstraction is important because Amazon can later become:

```
DataForSEOAmazonProvider → AmazonCreatorsAPIProvider
```

without changing the resolution service, database model or frontend.

7. Suggested Data Model

Field	Purpose
product_id	Internal PricePing product ID
marketplace	amazon / flipkart / myntra
external_product_id	ASIN or marketplace-specific product ID
title	Canonical product title
brand	Brand
model	Model/product number where available
variant_attributes	RAM, storage, size, color, generation, etc.
product_url	Canonical marketplace product URL
image_url	Primary product image
price	Latest resolved price
currency	INR
source_provider	DB / Flipkart API / DataForSEO Amazon / DataForSEO Shopping
last_verified_at	Timestamp of last successful validation
match_confidence	Internal confidence score

8. Matching Strategy

Use a two-stage matcher. Stage 1 is inexpensive candidate retrieval/filtering. Stage 2 validates product identity.

```
Candidate score = title semantic similarity + brand exact match + model exact match + variant match +
marketplace/domain match + identifier match
```

Recommended rules: an exact ASIN/product ID match should dominate all other signals; brand/model mismatches should strongly penalize a candidate; variant mismatches such as S24 vs S24 Ultra, 128 GB vs 256 GB, or 6 GB vs 8 GB should be treated as different products; title similarity ≥0.90 is a useful gate but should not be the sole decision rule.

9. Caching and Cost Optimization

The DB-first design is the main cost-control mechanism. External calls should be made only when the DB cannot satisfy the request or the user explicitly rejects the cached result.

Recommended cache keys:

• Marketplace + external product ID (strongest key)
• Marketplace + normalized canonical URL
• Marketplace + normalized title/model/variant
• Search-query cache with a controlled TTL for unresolved searches

After a successful external resolution, persist the result. The next user searching for the same product can receive the cached result without paying for another external request.

10. Error and Fallback Handling

If the selected provider fails, return a controlled 'could not resolve' state rather than an unverified product. For Amazon, DataForSEO is the primary temporary resolver. Later, Amazon Creators API becomes the preferred source after eligibility is achieved. For Myntra, Google Shopping is the selected discovery mechanism; if it returns no Myntra candidate, do not silently substitute another marketplace.

11. Why This Architecture

• **DB first:** fastest response, lowest cost, and increasingly accurate as the product catalog grows.

• **Marketplace-specific providers:** direct APIs are more reliable than generic web search when available.

• **Amazon → DataForSEO:** structured Amazon product data avoids the unreliable URL/image extraction seen with Tavily and bridges the period before Amazon Creators API access.

• **Flipkart → direct API:** strongest available source because PricePing already has the integration.

• **Myntra → Google Shopping/DataForSEO:** practical discovery route because a dedicated Myntra DataForSEO API is not part of this architecture.

• **Provider abstraction:** prevents vendor lock-in and makes the eventual Amazon API migration simple.

• **User rejection triggers external resolution:** preserves user control and avoids repeatedly serving an incorrect cached match.

• **Persist successful resolutions:** every lookup improves the DB and reduces future API cost.

12. End-to-End Example

Example: user searches for 'Samsung Galaxy S24 5G 128GB'.

```
1. Normalize query. 2. Search PricePing DB. 3. DB has a high-confidence Amazon product → return it.
4. User says 'Not this product'. 5. Mark the cached candidate as rejected for this request. 6. Route
to Amazon resolver. 7. DataForSEO Amazon search returns candidates. 8. Match
title/brand/model/storage and rank candidates. 9. Return the best candidate or ask the user to choose
if confidence is ambiguous. 10. Persist the validated product. 11. Future searches can now be served
from PricePing DB.
```

13. Evolution Roadmap

```
Phase 1 — Current MVP DB → Flipkart API / DataForSEO Amazon / DataForSEO Google Shopping Phase 2
Improve product matching, variant extraction, rejection feedback, caching and provider monitoring.
Phase 3 Amazon Creators API becomes available → replace DataForSEO Amazon as the preferred Amazon
provider. Phase 4 Add additional direct affiliate/product APIs where available.
```

14. Final Recommended Architecture

The key design principle is: **search internally first, use the best marketplace-specific external source only when necessary, validate before accepting, and store the validated result for reuse.**